
Daily Progress Report

Week 4_Day5: Week 4 Mini-Project

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_4 (Day 5)
Date	17 July, 2026

Day 5: Week 4 Mini-Project

1.1 Topics Studied

- Asynchronous API communication using the Fetch API and async/await
- Consuming third-party REST APIs (Open-Meteo Geocoding & Weather API, DummyJSON Quote API)
- Understanding JSON responses and extracting nested object properties
- Dynamic DOM manipulation based on API responses
- Browser localStorage for persisting favorite quotes
- ES6 Module syntax (import and export) for modular application development
- Separation of concerns through dedicated API, UI, storage, and main controller modules
- Error handling using try...catch and graceful fallback mechanisms

1.2 Task Description

Developed a dynamic **Weather & Quote Dashboard** that integrates multiple public APIs to provide real-time weather information and inspirational quotes. The application allows users to search weather conditions for any city, retrieve random motivational quotes, and save their favorite quotes using the browser's localStorage for persistent storage. The project was architected using ES6 modules to separate API communication, UI rendering, storage management, and application control into independent, maintainable files following modern JavaScript development practices.

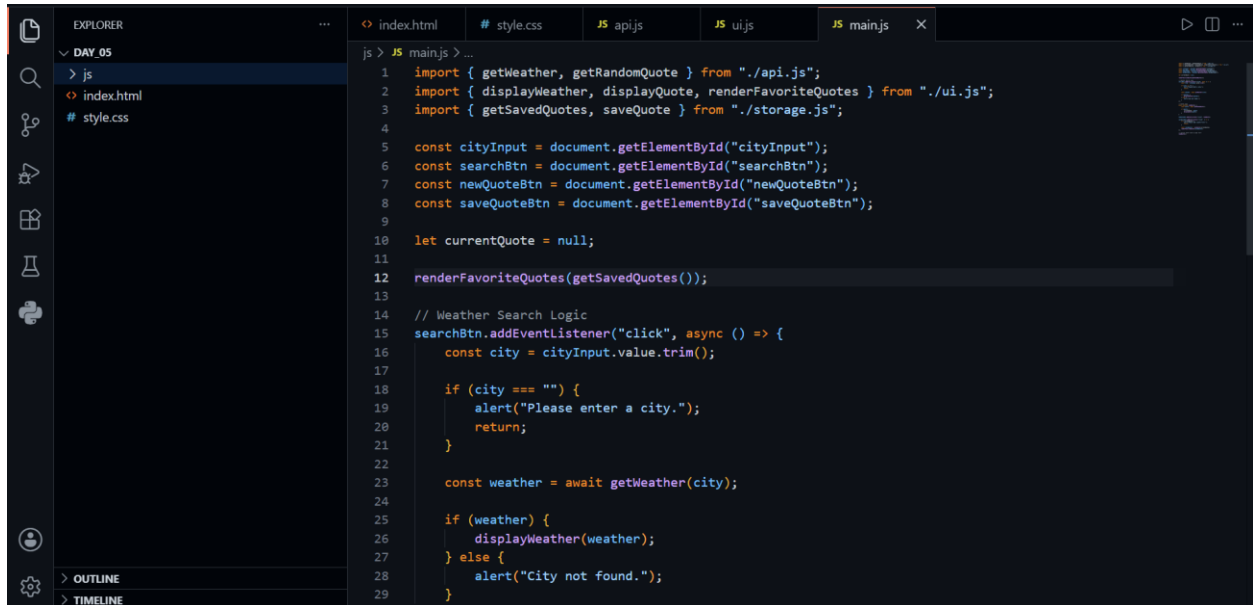
1.3 Implementation Details

- Designed a responsive dashboard interface consisting of dedicated weather, quote, and favorite quotes sections.
- Implemented asynchronous weather retrieval using the Open-Meteo Geocoding API to convert city names into geographic coordinates, followed by the Open-Meteo Weather API to fetch current weather information.
- Utilized the Fetch API together with `async/await` for clean asynchronous request handling and improved code readability.
- Processed JSON API responses to dynamically display city name, temperature, humidity, wind speed, weather description, and weather icons within the dashboard.
- Integrated the DummyJSON Quote API to automatically retrieve and display a random inspirational quote when the application loads.
- Implemented quote management functionality allowing users to save favorite quotes into `localStorage` using `localStorage.setItem()` combined with `JSON.stringify()`.
- Retrieved previously saved favorite quotes during application initialization using `localStorage.getItem()` and `JSON.parse()`, ensuring persistent client-side storage across browser sessions.
- Modularized the project into dedicated ES6 modules (`api.js`, `ui.js`, `storage.js`, and `main.js`), separating API communication, DOM manipulation, storage operations, and application orchestration to improve maintainability and scalability.
- Implemented structured error handling using `try...catch` blocks to gracefully handle invalid city searches, network failures, and API communication errors without interrupting the user experience.

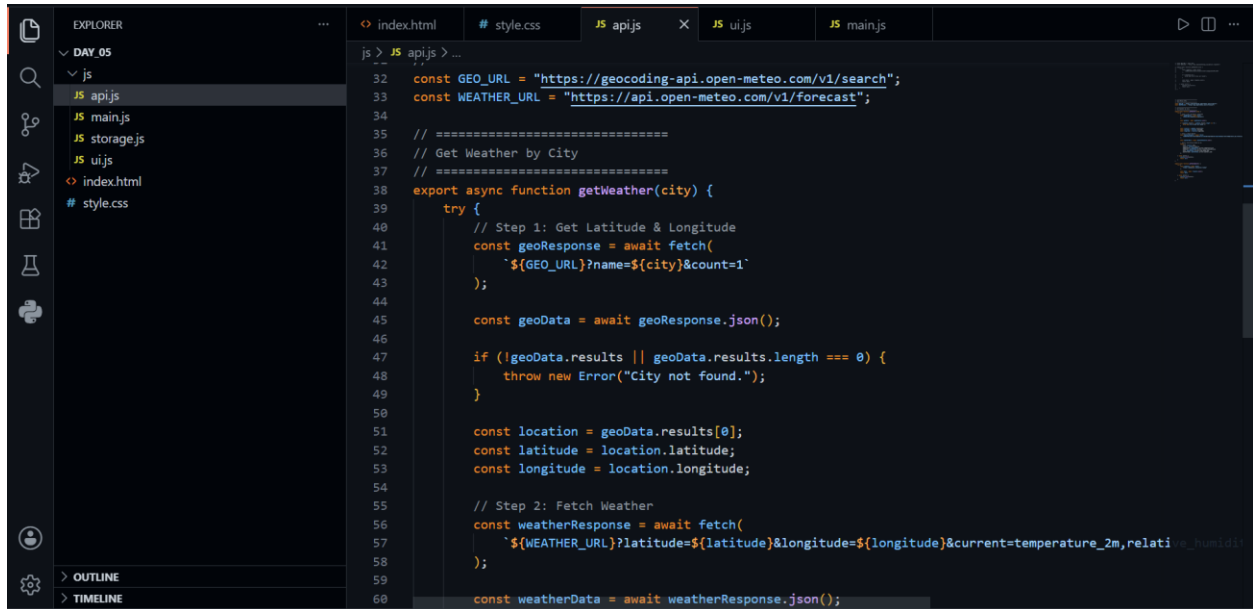
1.4 Outcome

Successfully developed a modern, API-driven dashboard that combines real-time weather information with motivational quotes while demonstrating practical implementation of asynchronous JavaScript programming and browser-based persistence. The integration of multiple REST APIs strengthened my understanding of asynchronous workflows, JSON processing, and dynamic DOM rendering. Furthermore, adopting ES6 modules established a clean, maintainable project architecture through proper separation of concerns, while utilizing `localStorage` enabled persistent management of favorite quotes across browser reloads. The completed application closely reflects modern front-end development practices used in real-world web applications.

Screenshots:



```
js > JS mainjs > ...
1  import { getWeather, getRandomQuote } from "../api.js";
2  import { displayWeather, displayQuote, renderFavoriteQuotes } from "../ui.js";
3  import { getSavedQuotes, saveQuote } from "../storage.js";
4
5  const cityInput = document.getElementById("cityInput");
6  const searchBtn = document.getElementById("searchBtn");
7  const newQuoteBtn = document.getElementById("newQuoteBtn");
8  const saveQuoteBtn = document.getElementById("saveQuoteBtn");
9
10 let currentQuote = null;
11
12 renderFavoriteQuotes(getSavedQuotes());
13
14 // Weather Search Logic
15 searchBtn.addEventListener("click", async () => {
16     const city = cityInput.value.trim();
17
18     if (city === "") {
19         alert("Please enter a city.");
20         return;
21     }
22
23     const weather = await getWeather(city);
24
25     if (weather) {
26         displayWeather(weather);
27     } else {
28         alert("City not found.");
29     }
30 }
```



```
js > JS apijs > ...
32 const GEO_URL = "https://geocoding-api.open-meteo.com/v1/search";
33 const WEATHER_URL = "https://api.open-meteo.com/v1/forecast";
34
35 // =====
36 // Get Weather by City
37 // =====
38 export async function getWeather(city) {
39     try {
40         // Step 1: Get Latitude & Longitude
41         const geoResponse = await fetch(
42             `${GEO_URL}?name=${city}&count=1`
43         );
44
45         const geoData = await geoResponse.json();
46
47         if (!geoData.results || geoData.results.length === 0) {
48             throw new Error("City not found.");
49         }
50
51         const location = geoData.results[0];
52         const latitude = location.latitude;
53         const longitude = location.longitude;
54
55         // Step 2: Fetch Weather
56         const weatherResponse = await fetch(
57             `${WEATHER_URL}?latitude=${latitude}&longitude=${longitude}&current=temperature_2m,relative_humidity`
58         );
59
60         const weatherData = await weatherResponse.json();
61     } catch (error) {
62         console.error("Error fetching weather data:", error);
63     }
64 }
```

Screenshots:

